

Практична робота №6

Виконали студенти групи ІП-44 Кулагін Дмитро та Комін Іван

Application: CreateSpace (Professional Space/Studio Booking for Creators)

Platform: Web

Тема: Забезпечення відмовостійкості та надійності системи

Мета: Навчитись передбачати потенційні проблеми системи на етапі формування технічних вимог, та приймати рішення, щодо їх коректної обробки.

Завдання:

1. Скласти список ключових потенційних відмов системи.
2. Вибрати по 1 проблемі на студента, та описати 2-3 варіанти рішення цих проблем.
3. Обрати рішення проблеми, та обґрунтувати з точки зору бізнесу (врахувати пріоритети, час виконання, ресурси, вартість і т.д.).
4. Описати діаграму послідовностей (sequence diagram) для проблем.
5. Вкажіть, які патерни використовуються.

Потенційні відмови системи

1. Відмови міжсервісної взаємодії

У нашій мікросервісній архітектурі певні операції вимагають миттєвої відповіді для продовження бізнес-процесу.

- **Таймаут запиту при розрахунку чека (Billing → Inventory):** Під час оформлення замовлення Billing Service звертається до Inventory Service для отримання актуальної ціни та підтвердження доступності обладнання. Якщо сервіс інвентарю не відповідає, розрахунок фінальної суми стає неможливим.
- **Відмова сервісу ідентифікації (Identity Service):** Оскільки API Gateway (YARP) валідує JWT токени через цей сервіс для кожного вхідного запиту, його недоступність заблокує доступ до всіх функцій платформи для авторизованих користувачів.

2. Відмови при роботі з базами даних

Ми використовуємо патерн **Database per Service**, що ізолює помилки БД одного домену від інших, але створює ризики всередині сервісу.

- **Вичерпання пулу підключень:** У пікові години бронювання Scheduling DB може перестати приймати нові запити на запис слотів. Це призведе до неможливості створити нове бронювання, хоча пошук студій (через репліки читання) продовжуватиме працювати.
- **Помилка транзакції при блокуванні слота:** Якщо під час запису статусу "Locked" стається збій, система може допустити **double-booking**, що прямо порушує наші нефункціональні вимоги.

3. Відмови системи обміну повідомленнями

RabbitMQ є фундаментом нашої Event-driven архітектури та забезпечує Eventual Consistency.

- **Неможливість публікації івенту (Publish Failure):** Після успішного запису бронювання в БД Scheduling Service намагається надіслати івент BookingConfirmedEvent. Якщо брокер недоступний, ланцюжок асинхронних дій (сповіщення клієнта, оновлення кешу білінгу) обривається.
- **Переповнення черг (Queue Overflow):** Якщо Notification Service не встигає обробляти повідомлення про відправку SMS, черга в RabbitMQ почне споживати надмірну пам'ять, що може призвести до падіння всього вузла брокера.

4. Відмови при взаємодії з зовнішніми системами

Наша платформа критично залежить від стабільності сторонніх API.

- **Збій Google Calendar API:** Помилка при отриманні вебхуків або виконанні поллінгу для синхронізації доступності слотів. Наслідок — відображення в CreateSpace слотів, які вже зайняті в особистому календарі власника студії.
- **Відмова Twilio API:** Notification Service отримує івент із черги, але не може надіслати SMS через технічні проблеми на стороні Twilio. Повідомлення про підтвердження бронювання не доходить до користувача.

Назва відмови	Рівень критичності	Вплив на бізнес та користувача	Технічний наслідок	Ймовірність
Відмова Identity Service	Critical	Повна зупинка платформи; користувачі не можуть увійти або забронювати простір.	API Gateway відхиляє всі запити через неможливість валідації JWT.	Низька
Таймаут Billing → Inventory	High	Пряма втрата доходу; користувач бачить помилку на етапі оплати послуг.	Переривання синхронного ланцюжка розрахунку вартості в ASP.NET Core.	Середня
Збій RabbitMQ (Broker Down)	High	Втрата узгодженості даних; сповіщення не надсилаються, календарі не оновлюються.	Розрив асинхронних зв'язків між мікросервісами системи.	Низька
Відмова Google Calendar API	High	Ризик подвійних бронювань (double-booking); значні репутаційні	Припинення синхронізації зовнішніх часових слотів у реальному часі.	Середня

		втрати для платформи.		
Збій Twilio API (SMS)	Medium	Погіршення UX; користувачі не отримують миттєвих SMS-підтверджень.	Накопичення повідомлень у черзі або їх переміщення до Dead Letter Queue.	Середня
Вичерпання пулу підключень БД	Medium	Тимчасова неможливість створити нове бронювання в SQL-базі у пікові години.	Відмова сервісу планування у записі нових транзакцій до PostgreSQL.	Середня

Обрані проблеми

Проблема 1

Відмова: Таймаут або помилка синхронного HTTP-запиту від Billing Service до Inventory Service під час формування розрахунку.

- **Варіант 1: Патерн Circuit Breaker.** Система відстежує кількість помилок від сервісу інвентарю. Якщо поріг перевищено, шлюз розмикає коло, миттєво повертаючи помилку наступним запитам без спроби з'єднання, що дозволяє сервісу "відпочити" та відновитися.
- **Варіант 2: Fallback до кешованих даних.** У разі недоступності основного сервісу, Billing Service звертається до власного кешу, де зберігаються останні відомі ціни на обладнання та студії. Це дозволяє завершити транзакцію, хоча ціна може бути неактуальною на кілька хвилин (Eventual Consistency).
- **Варіант 3: Fail-fast з інформуванням користувача.** Система миттєво повертає помилку на UI з поясненням, що сервіс розрахунку тимчасово недоступний, та пропонує зберегти замовлення у чернетку для спроби оплати пізніше.

Обране рішення для Проблеми 1

Рішення: Гібридний підхід — Патерн **Fallback** до кешу у поєднанні з **Circuit Breaker**.

Обґрунтування з точки зору бізнесу:

Параметр	Опис та значення
Пріоритет	Найвищий. Будь-яка перешкода на етапі оплати напряду зменшує прибуток платформи.
Час виконання	Середній (2–3 дні). Налаштування бібліотеки Polly для .NET та логіки запису цін у Redis при кожному оновленні інвентарю.
Ресурси	Потрібен один розробник та існуючий інстанс Redis.
Вартість	Низька. Ми використовуємо вже наявну інфраструктуру кешування.
Бізнес-цінність	Ми забезпечуємо безперервність продажу. Навіть якщо сервіс інвентарю «впав», клієнт завершує бронювання за останньою актуальною ціною.

Проблема 2

Відмова: Помилка доставки повідомлення через зовнішній API Twilio після успішного завершення бронювання.

- **Варіант 1: Асинхронні повтори (Retry Pattern) через чергу RabbitMQ.** Якщо Twilio повертає помилку, повідомлення не видаляється з черги, а відправляється на повторну спробу з експоненціальною затримкою (через 1, 5, 15 хвилин). Це нівелює вплив короточасних збоїв на стороні провайдера.
- **Варіант 2: Використання альтернативного каналу (Email/In-app).** При неможливості надіслати SMS, система автоматично перемикається на відправку

електронного листа або Push-сповіщення у застосунок, щоб гарантувати, що клієнт отримає інформацію про підтвердження.

- **Варіант 3: Патерн Dead Letter Queue (DLQ).** Після вичерпання ліміту спроб повідомлення переміщується до спеціальної черги "битого" трафіку. Адміністратор отримує сповіщення та може вручну перевірити статус або повторити відправку після усунення проблеми.

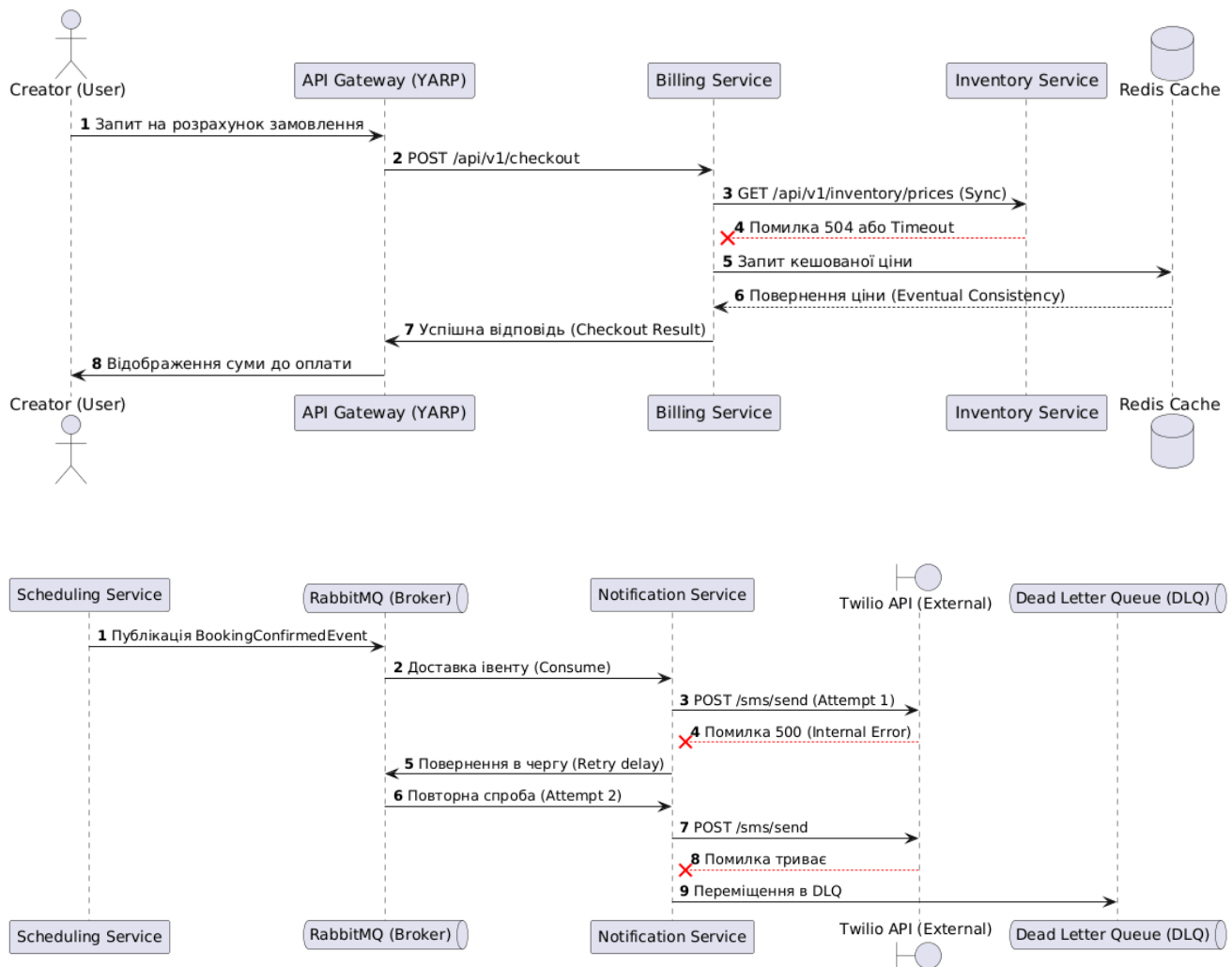
2. Обране рішення для Проблеми 2

Рішення: Retry Pattern з експоненціальною затримкою та використанням Dead Letter Queue.

Обґрунтування з точки зору бізнесу:

Параметр	Опис та значення
Пріоритет	Високий. Своєчасне підтвердження бронювання в SMS — це запорука довіри користувача до сервісу.
Час виконання	Короткий (1 день). Конфігурація черг у RabbitMQ та додавання логіки повторів у споживача.
Ресурси	Потрібен один розробник, який працює з RabbitMQ та .NET Background Tasks.
Вартість	Мінімальна. Відсутність потреби купувати підписку на другого резервного SMS-провайдера.
Бізнес-цінність	Ми гарантуємо доставку сповіщення «At-least-once». Якщо Twilio тимчасово недоступний, система сама «дотисне» відправку пізніше.

Sequence diagram



Опис патернів, що використовуються на діаграмах:

- **Circuit Breaker:** Використовується в Проблемі 1 для запобігання перевантаженню несправного сервісу та миттєвого перемикання на запасний план.
- **Fallback (Запасний варіант):** Використання Redis як альтернативного джерела даних, коли основне джерело недоступне.
- **Retry with Exponential Backoff:** Автоматичне повторення запиту до зовнішнього API з інтервалами, що збільшуються, для обробки тимчасових збоїв.
- **Competing Consumers:** Використовується для паралельної обробки сповіщень декількома інстансами Notification Service.
- **Dead Letter Queue (DLQ):** Кінцева точка для повідомлень, які неможливо обробити автоматично, що гарантує відсутність втрати даних.

Висновок: У результаті виконання шостої практичної роботи було проведено комплексний аналіз відмовостійкості системи **CreateSpace**, що дозволило ідентифікувати критичні точки відмови у синхронних та асинхронних взаємодіях. Шляхом впровадження патернів **Circuit Breaker** із **Fallback** до кешу Redis для сервісу білінгу, а також механізму **Retry** із **Dead Letter Queue** для сповіщень через Twilio, було забезпечено стабільність бізнес-процесів навіть за умов тимчасової недоступності окремих компонентів чи зовнішніх API. Обрані рішення є економічно обґрунтованими, оскільки вони мінімізують ризики втрати доходу та покращують UX без значного збільшення витрат на інфраструктуру, що підтверджує готовність архітектури до роботи в умовах реальних мережевих збоїв.